Backend Workshop

Client–Server, HTTP, APIs, and Databases

2026-04-07 • Beginners to early-intermediate developers

Catalog

1 Workshop outcomes

2 Backend fundamentals

3 Client–server & request lifecycle

4 Internet & HTTP

5 Methods, status codes & payload formats

6 Debugging with DevTools

7 APIs & REST design

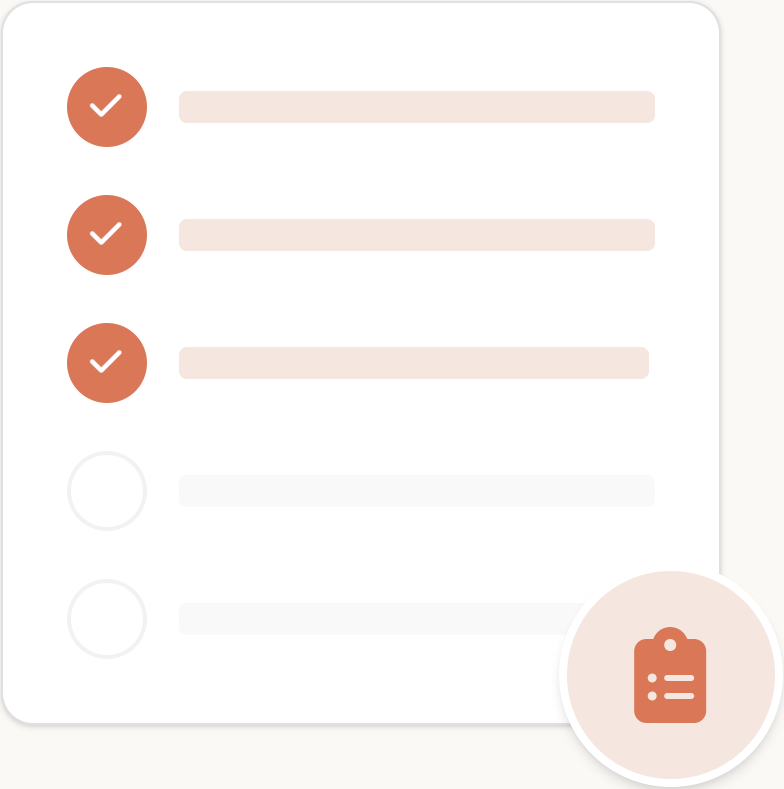
8 Architectures, tech stack & data storage

© 2026 Backend Development Workshop

What You Will Learn

By the end of this workshop, you will be able to:

- Understand how web and mobile applications communicate with backend systems
- Explain the request–response lifecycle
- Use browser DevTools to inspect and debug API calls
- Design and interact with REST APIs
- Understand HTTP methods and status codes
- Work with data formats (JSON, XML)
- Distinguish server-rendered apps vs modern SPA architectures
- Build a basic backend service backed by a database
- Explain (at a high level) how backend systems are deployed and scaled



© 2026 Backend Development Workshop

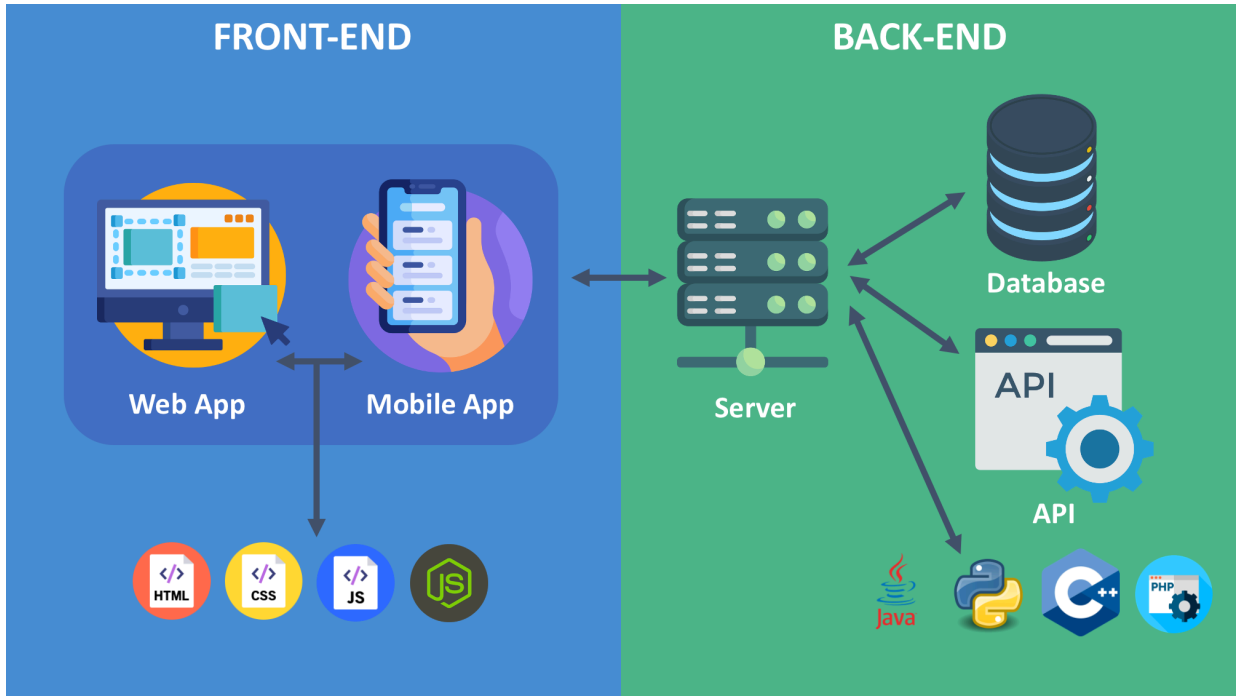
What is Backend?

Definition: The backend is the server-side part of an application that powers functionality and data.

Components: Client → API Layer → Server App → Database

Handles:

- Data storage & retrieval
- Business logic (rules, validations)
- Authentication & authorization
- API communication with clients



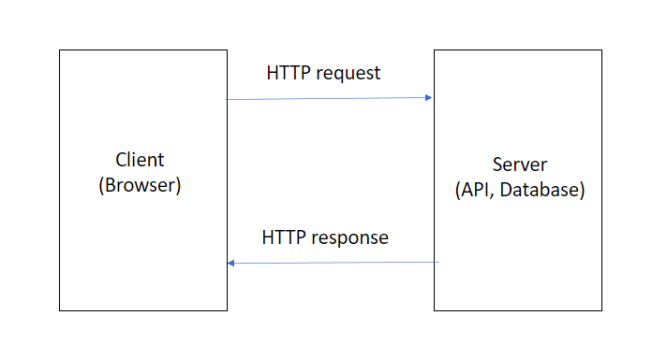
Client vs Server

Client (Browser / Mobile app)

- Sends requests (e.g., fetch profile, submit form)
- Renders UI and reacts to responses

Server

- Receives requests
- Executes logic
- Reads/writes data
- Sends responses



Key concept: Communication happens over the internet using HTTP/HTTPS

© 2026 Backend Development Workshop

The Lifecycle of a Web Request

Request = input

(what the client asks for)

Response = output

(what the server returns)

© 2026 Backend Development Workshop

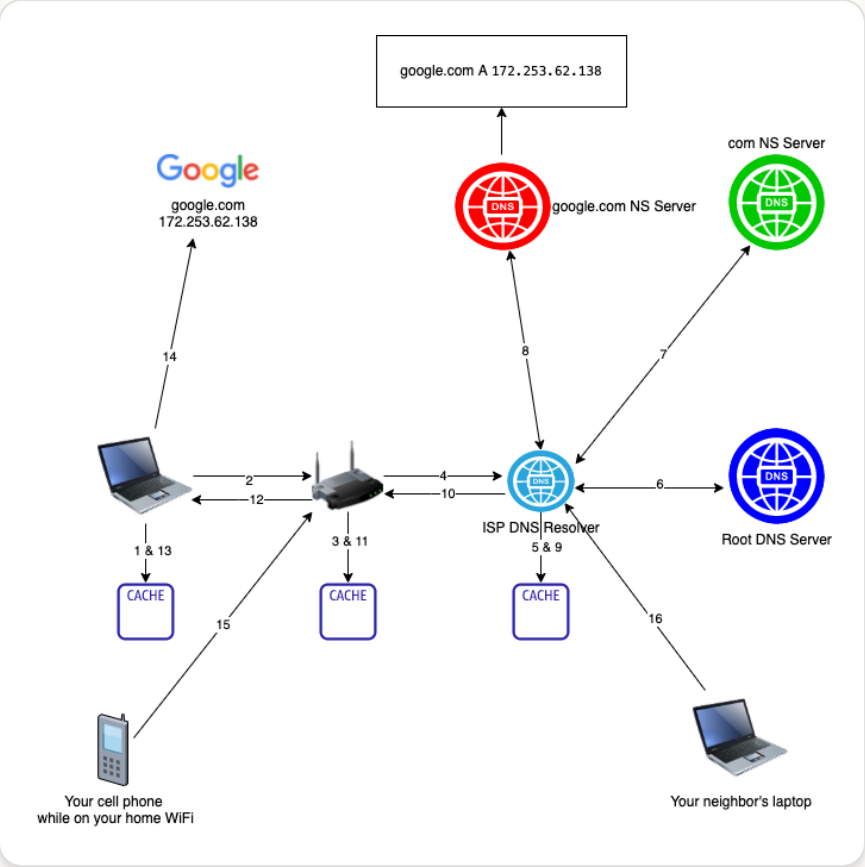
How the Internet Works

Key concepts:

- Domain → IP (DNS lookup)
- Communication over HTTP/HTTPS
- Servers are addressed by IP + port

Ports:

80	→ HTTP	443	→ HTTPS



What an HTTP Message Contains

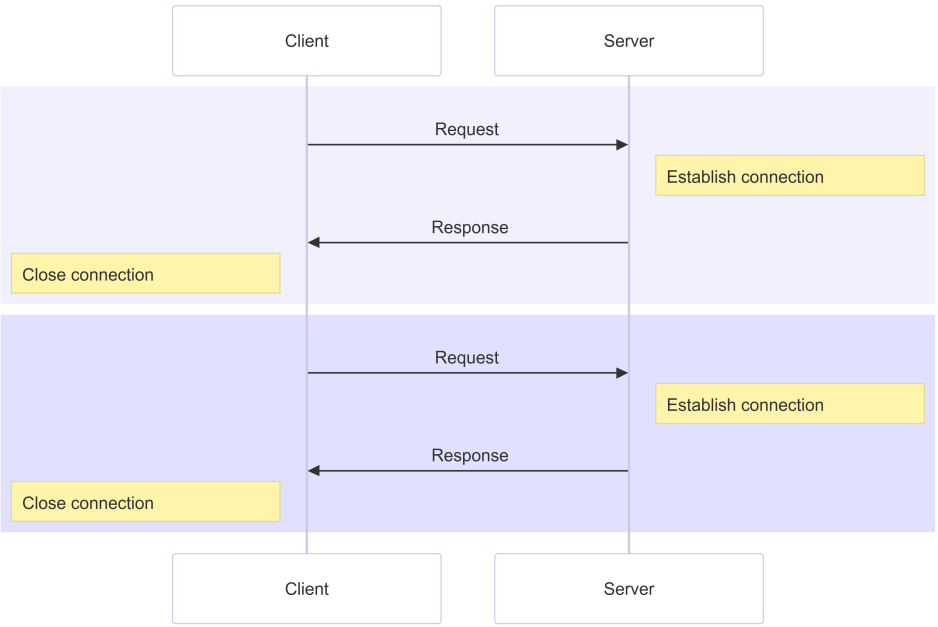
Request Structure

- Method
- URL
- Headers
- Body *(optional)*

Response Structure

- Status code
- Headers
- Body

Stateless: Each request is independent; the server does not remember prior requests by default.



Verbs That Define the Action

GET

Retrieve data

```
e.g., GET /users
```

POST

Create resource

```
e.g., POST /orders
```

PUT

Update/replace resource

```
e.g., PUT /users/123
```

PATCH

Partial update

```
e.g., PATCH /users/123
```

DELETE

Remove resource

```
e.g., DELETE /orders/1
```

© 2026 Backend Development Workshop

How Servers Communicate Outcomes

2xx
Success

4xx
Client errors

5xx
Server errors

Common Codes

200	OK	401	Unauthorized
201	Created	404	Not Found
400	Bad Request	500	Internal Server Error

© 2026 Backend Development Workshop

What Data Looks Like Over the Wire

JSON (standard, lightweight)

```
{
  "user": "yousri",
  "email": "test@mail.com"
}
```

XML (legacy, verbose)

```
<user>
  <name>yousri</name>
</user>
```

- Used for API communication
- Serialization: object → text
- Deserialization: text → object

Browser DevTools

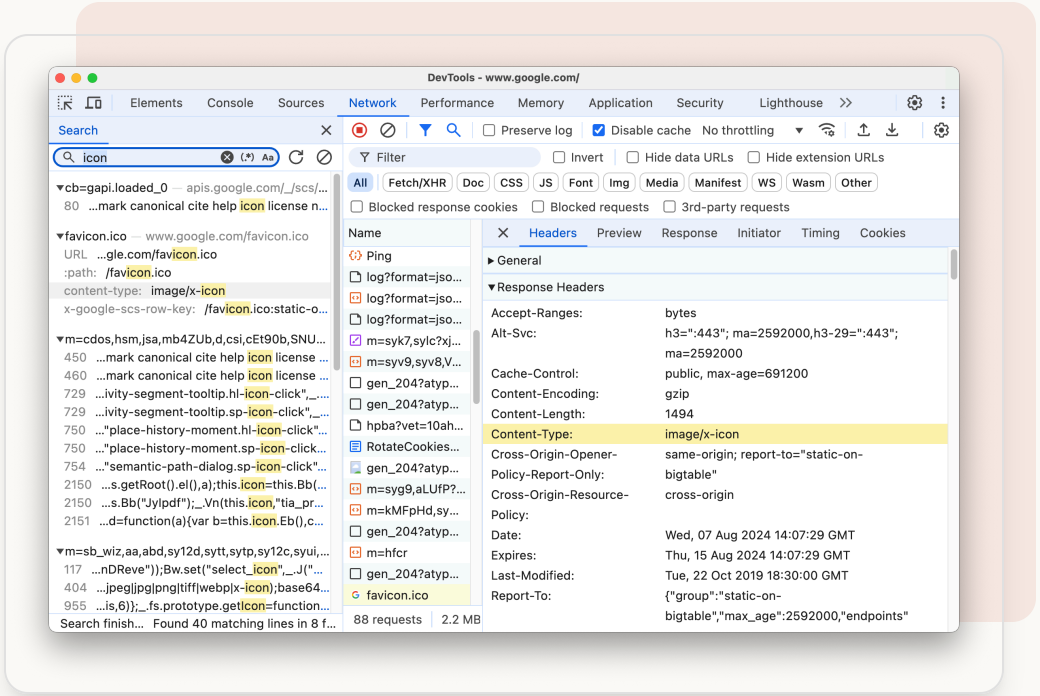
Inspect and Debug API Calls

What to look at

- Request URL (endpoint)
- Request headers
- Request payload (if any)
- Response body (data)
- Status code

How to use it (Demo flow)

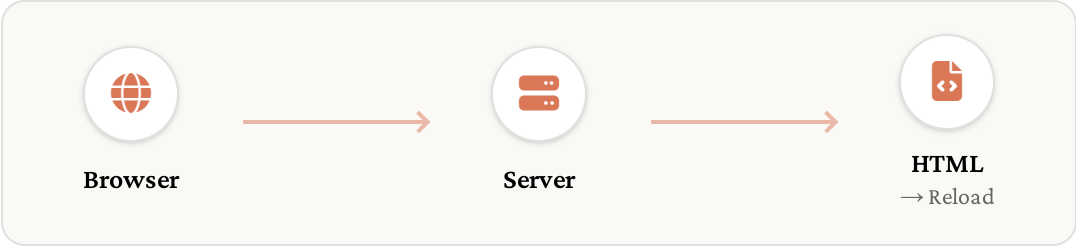
1. Open any site
2. Trigger an action
3. Select an `/api/...` request
4. Inspect details



SPA vs Traditional Applications

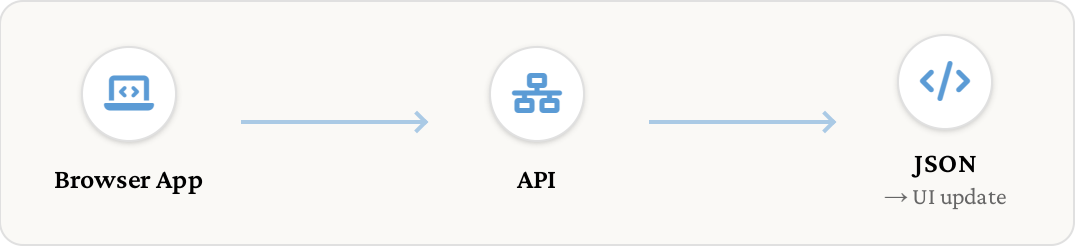
Traditional (server-rendered, e.g., PHP)

- Server returns full HTML pages
- Each request reloads page
- Backend handles rendering



SPA (React / Angular)

- Frontend loads once
- Backend returns JSON
- UI updates dynamically



Key difference: Server → HTML (traditional) vs Server → JSON (SPA)

© 2026 Backend Development Workshop

Common Server-Side Tools

Languages

- JavaScript (Node.js)
- Python
- Java
- Ruby

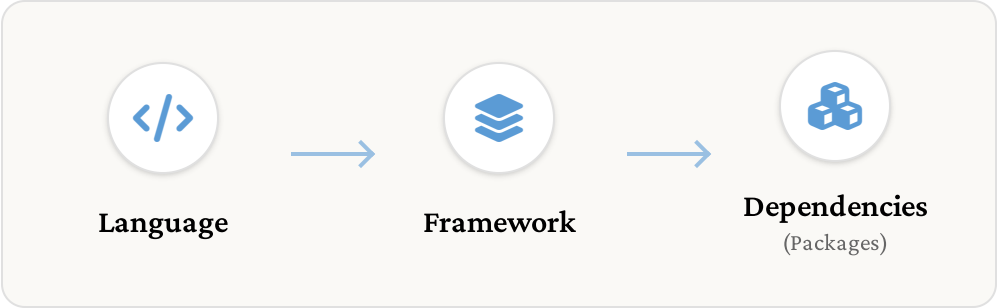
Frameworks

- Express (Node.js)
- Django / FastAPI (Python)
- Spring (Java)

Package Managers

- NPM
- PIP
- Maven

SIMPLE STACK ARCHITECTURE



© 2026 Backend Development Workshop

Databases

Where Data Lives

Purpose

Store and manage application data securely and efficiently for retrieval and updates.

Types

SQL

Relational, structured tables

PostgreSQL | MySQL

NoSQL

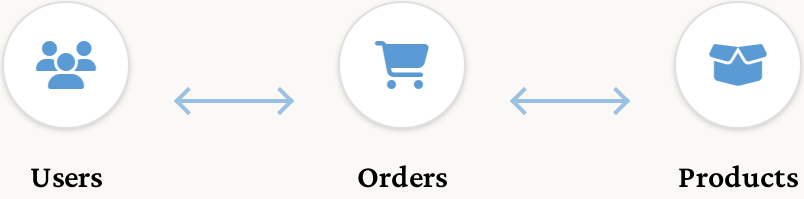
Flexible, document-based

MongoDB

Examples of Stored Data

- Users
- Orders
- Products

MINI DATA MODEL



© 2026 Backend Development Workshop

Let's Practice

Q&A